Acknowledgement

We are grateful to our respected Professor Arnab Chakrabarty, who gave us the opportunity to work on such an interesting topic. He has also been instrumental in guiding us through this project successfully. With his wisdom and knowledge, we were able to complete this report with ease under his supervision which was a very enriching experience for all of us!

Contents

1 Objective

Chroma keying deals with removal of background from foreground. Now in case of still pictures, it is relatively a lot easier to specify which points belong to background and which belong to foreground. However, in case of videos there are plenty of frames so we need to come up with a general algorithm that works for all frames. So, while we work with the special case images, we need to keep in mind that the criterion for checking whether an image is in background should be independent of the position coordinates of an object as they may change frequently in a video.

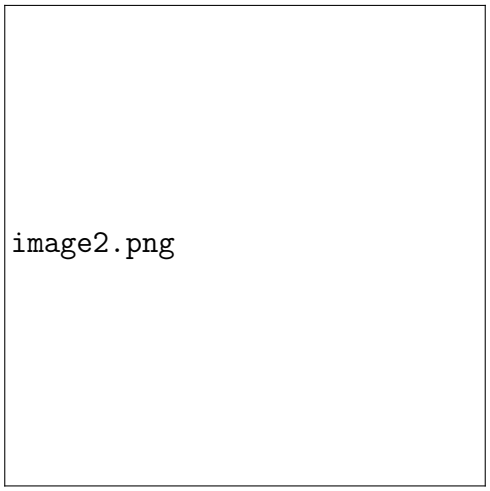
Key Objectives:

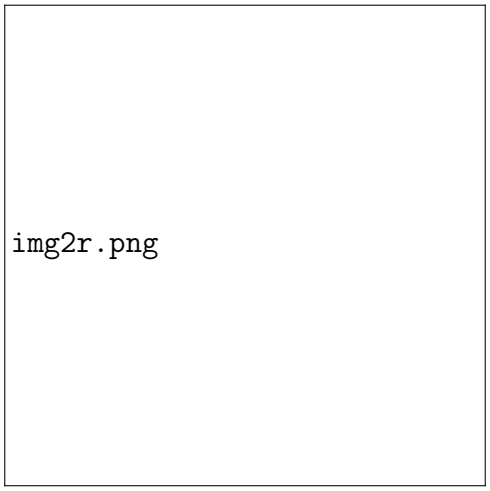
1. To develop simple algorithms to separate background from foreground and implement them. Then documenting the problems in each algorithm and trying to improve upon them using the results in class.
2. Finalising the best possible algorithm among the ones in point 1 and trying to inspect the extreme cases like fluffy hair,transparent objects etc and trying to find minor corrections to the algorithms.
3. Finding a suitable error approximation technique to check the viability of our algorithm.
4. To reduce the error and try to include as many extreme cases as possible.

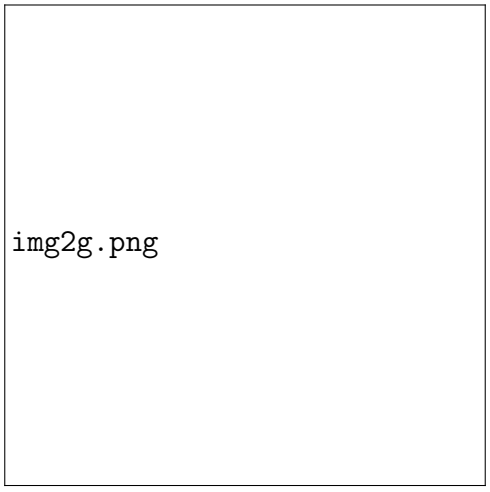
.

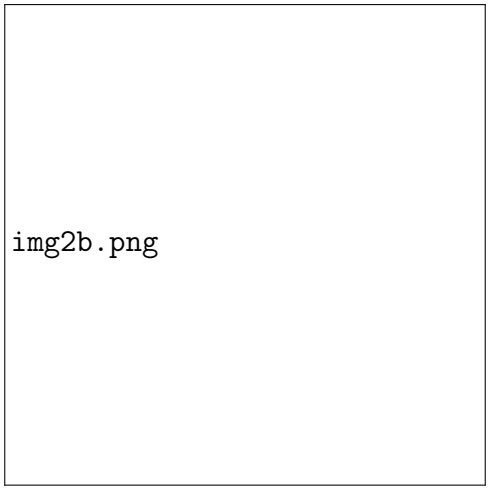
2 Exploring Different Approaches

2.1 The Histogram Screening



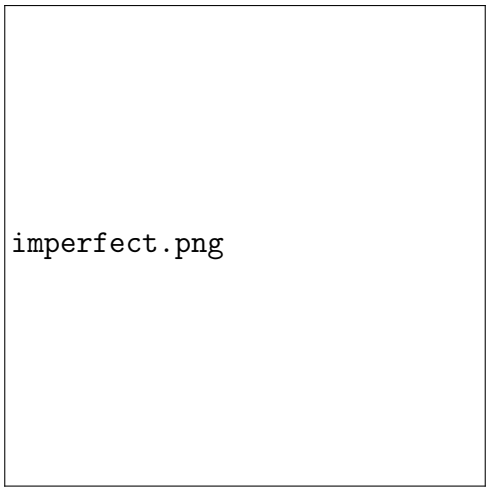





img2b.png

2.1.1 Algorithm

We observe from the histograms of most of our images (one image given for illustration) that some collected bars are very high and the rest are very low compared to them and the height abruptly reduces at a certain point. As variation in the background will be lower, the abruptly high bars represent the points in background for e.g for this particular image , we get the following output.



2.1.2 Drawbacks

1. Clearly finding an appropriate bound is very difficult as it is difficult to quantify the abrupt change.

2. Even after finding the best possible bound , it will fluctuate as the images change so not a practical solution for a video.

3. It is always at the risk of including large uniform patches patches of foreground as background while excluding a small irregular patch of background.

2.2 A Slightly Smarter Approach

2.2.1 Algorithm

Our previous approach doesn't account for the fact that in the background the green component will be higher and the red and blue components will be lower. Using this fact we can give a smarter bound for e.g red and blue component < 0.5 and green component > 0.35 works pretty well.

For example in the following images, the bound works pretty well.

2.2.2 Image Transformation

INPUT IMAGES	OUTPUT IMAGES	INPUT IMAGES	OUTPUT IMAGES
image1.png	im1.png	image5.png	im5.png
image2.png	im2.png	image6.png	im6.png
image3.png	im3.png	image7.png	im7.png
image8.png	im8.png	image15.png	im15.png

INPUT IMAGES	OUTPUT IMAGES	INPUT IMAGES	OUTPUT IMAGES
image9.png	im9.png	image13.png	im12.png
image10.png	im10.png	image14.png	im14.png
image11.png	im11.png	image12.png	im27.png

2.2.3 Drawbacks

1. Some images still have noticeable error. Also as we have found the bound completely by hit and trial, so we do not know the factors on which it depends.

2. The error here is not manageable because it will be different in each frame and the error region has to be specified.

3. We have also failed to quantify a parameter that depends on the green screen.

3 A Different Implementation

Instead of taking bounds for red,blue and green components why don't we bound the ratios g/r and g/b , we can say they will be surely more than 1 for the background as it is green and by observation we have found out that if g/r and g/b values are greater than 1.2(call it the supper bound) then, it works quite well.

Also, we have noticed that we get the best possible images by varying the supper bound between 1 and 2 i.e we check each image for supper bounds in 1,1.1,1.2,...,2 . We also get 1.2 as the supper bound most of the time. Start implementing with 1.2, if foreground is removed , a value greater than 1.2 works and if background is left behind(very rare case), a value less than 1.2 works

INPUT IMAGES	BOUND	OUTPUT IMAGES	INPUT IMAGES	BOUND	OUTPUT IMA
image1.png	1.2	im16.png	image5.png	1.2	im20.png
image2.png	1	im17.png	image6.png	1.2	im21.png
image3.png	1.2	im18.png	image7.png	1.2	im22.png
image8.png	1.2	im23.png	image15.png	1.5	im30.png

INPUT IMAGES	BOUND	OUTPUT IMAGES	INPUT IMAGES	BOUND	OUTPUT IMA
image9.png	1.2	im24.png	image13.png	1.2	im28.png
image10.png	1.2	im25.png	image14.png	1.5	im29.png
image11.png	1.2	im26.png	image4.png	1.5	im19.png
image12.png	1.3	im27.png			

3.1 Drawbacks Before Finer Inspection

1. We still haven't been able to find the green screen parameter for the images though the supper bound may be a parameter which we will explore in the next section.
2. Though the error is not noticeable(if not very minutely observed) , we haven't been able to quantify it owing to lack of sophisticated machinery to calculate exact no of points in background and foreground.
3. After implementing with 1.2, it may happen that some background is left behind and some foreground is removed, though that didn't occur yet.
4. Finding the super bound for each frame is a tedious process but this time by scrutinising the pictures, we may be able to correlate the type of picture with its super bound.

3.2 Finer inspection

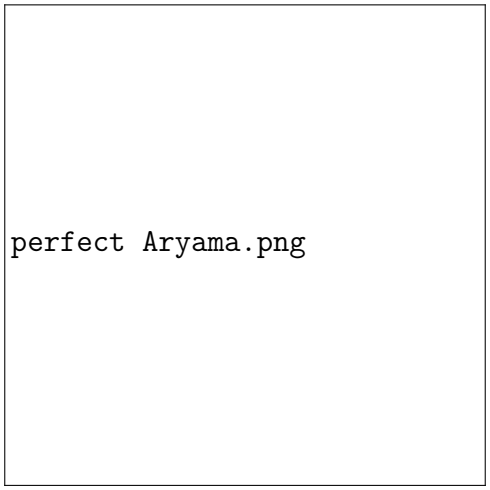
3.2.1 The Super Bound

Our algorithm predicts that points whose g/r and g/b value > super bound is in background. Now if we take our blank green screen and find the min of g/r values , it would be 1.289340 , similarly the min of g/b values would be 1.231884. Now we will take 90 percent (1.160406, 1.1086956 (call it b-bound)) of these to account for slight changes to the background owing to the fore ground. After applying these bounds to our pictures, all points in our background should be removed at the cost of some of our foreground (like in Adeesh's image, Mayank's image, wherever the observed super bound is greater than 1.1086956)

Keypoint

The minimum ratio does not change much because any change that the foreground brings to the background is percentage change in the three components that cancels out when we take the ratio. This also tells us this is actually an intrinsic property of the green screen and we can use it for rating it unlike the range of each component that may change drastically (for example the minimum green component of green screen is 0.5 and we have located a point with g component 0.29)

To confirm our hypotheses that all points in background are removed we only need to test it for Aryama's image as that is the only case where the super bound is < 1.1702898. So here is the image after applying the new bound where clearly all points of background have been cleaned off.



Possible Malfunctioning

If the green shade of the green screen undergoes some abnormal change between the frames and the cutoff of 90 percent still leaves some of the background intact.

Proposed solution and measures to reduce malfunction

Mark some key frames between which the lighting does not abruptly change and use corresponding bounds for frames lying in same interval. Also steps should be taken while filming to ensure minimum interaction with green screen and there should be a light to remove the shadow.

3.2.2 The Green Screen Rating

We will rate the green screen on the basis of its g-bound. Observe that if the g-rating = min(b-bound) > 1.5 , no foreground would have been removed. So, that would have been a better green screen for our set of images than our original green screen. In general, if our g-rating is higher, the lesser the adjustment we have to do to bring back our deleted foreground.

But is the g-rating parameter sufficient to rate a green screen?

NO, we also need to know the variation of the actual g-component of the green screen. The smaller that is ,the better the green screen is. Also we have used only the b-bound information in our algorithm but that is not enough as our algorithm perceives r=1, b= 1, g=5 (in 0-255 scale) as a point in the background but it is clearly black. Now we can take two approaches from here.

- To quantify what exactly does our brain perceive as **green** and the type of green our green screen is.

- To explicitly decode the points where g/r, g/b components are g-bounded , but the pixel is not **green** and hence can lie in the foreground.

4 The Two Approaches

If we find substantial results for approach 1, we can generalise it to other coloured screens and we can even separate the type of green of foreground and the type of green in background but the possible drawbacks are quantifying what is green and types of green is a very hefty task and applying those to every point in a frame increases the time complexity drastically. This would be more useful in a very sophisticated environment.

If we find substantial results for approach 2, this would give us good results without increasing the time complexity too much and would be perfect for environments the background is always green and the foreground does not contain green in any shade.

A suitable approach would be to mark the frames where the green object appears and apply approach 1 only to those frames.

For, both approaches , we need to find out "What is green?" i.e those points whose g>r and g>b but still not green.
To be continued.....

4.1 What is Green

We will be using 0-255 scale all throughout.

- For a colour to be perceived as green, clearly the g/r and g/b components will be > 1, that is a necessary condition. From here we get an important condition about rating our green screen i.e ming-bound should be greater than 1, in any key frame.

- If a particular (r,g,b) is perceived as green then, (r',g ,b') will be perceived as green provided r'<=r and b'<=b. So, for each g= 0 to 255, we need to find the maximum (r,b) that works.

- We used custom colours from paint to observe that if g<35(r,b<g), then it will be black not green even if the ratio is g bounded. We have taken some liberty at the boundary "green" points. There is always a trade off between the amount of variation brought about by foreground to the background and up to what shade of green is allowed in the foreground.

- So, basically for each g=36 to 255, we can find x such that if r=b<=x, the pixel is green and for each x<=y<=g, we need to find z(<=x) such that if max(r,b)= y and min(r,b) <= z, the pixel is green. The above is a tedious task but can be done. We are basically transferring our understanding of the colour green to the computer.
- Once we find that out, we can clearly say what shades of green(wherever g> b and g>r) should not appear in our background.
- Once we do this, approach 2 is immediate, we will just remove the points the ratios are g-bounded and "green".
- We will give all the boundary colours and state that these colours and colours greener than this can not appear in the foreground.

5 Is Error The Only Way?

We propose a different model for checking how well our algorithm works rather than calculating the no of points in the background that are not removed or no of points of foreground that have been removed because that number would be extremely small and largely depend on our set of images. Instead we are going to specify in terms of colours like for a particular green screen (we have not completely parameterised our green screen yet), what colours should not appear in foreground.

We also aim to grade our foreground with respect to our background (for approach 1) and find a threshold beyond which chroma keying is not possible.

To be continued....